SKN35 · LLM 연동 내외부 문서 기반 질의응답 시스템

# LangChain 버전 마이그레이션 도우미

옛 버전 코드를 원하는 버전에서 돌아가는 코드로, 근거와 검증까지 함께

SKN35 3rd 4Team · 발표 [발표자 이름] · 2026-10-15

문제

# ChatGPT가 준 LangChain 코드, 실행하면 깨집니다

```
# LLM이 흔히 주는 코드

from langchain.chat_models import ChatOpenAI
from langchain.chains import LLMChain

---

# LangChain 1.4.4에서 실행 결과

ImportError: cannot import name 'ChatOpenAI'
ModuleNotFoundError: No module named 'langchain.chains'
```

## 첫 줄부터 에러

LangChain 1.0에서 패키지가 나뉘고, 옛 체인과 에이전트가 빠졌습니다.

## 더 큰 문제

LLM은 자기 답이 틀렸다는 것을 **모릅니다.**

원인: 버전 지체 환각

# 인터넷에는 옛 버전 코드가 더 많습니다

LLM은 데이터가 많은 쪽을 따라갑니다. 라이브러리는 빠르게 바뀌지만, 학습 데이터 속 코드는 대부분 과거 버전입니다.

**42.55%**

문서가 부족할 때, LLM 코드 중 목표 버전에서 실행되는 비율

When LLMs Lag Behind, 2026

**66.36%**

구조화된 문서를 줘도 실행되는 비율

When LLMs Lag Behind, 2026

**48~51%**

최신 상용 모델의 버전 지정 코드 생성 성공률

GitChameleon 2.0, 2025

해결

# 근거로 고치고, 검사로 한 번 더 확인합니다

| 구분 | 일반 LLM | 우리 챗봇 |
| --- | --- | --- |
| **근거** | 학습 데이터 (옛 버전이 다수) | import · 메서드 대응표 + 최신 공식 문서 검색 (RAG) |
| **확인** | 없음 | verifier가 8가지 검사 (문법, import 존재, 남은 옛 import 등) |
| **틀렸을 때** | 모름 | 실패 이유로 근거를 다시 찾고, 틀린 근거는 빼고 다시 고침 (최대 3회) |
| **답변** | 코드만 | 고친 코드 + 바뀐 줄마다 근거 ID + 시도 횟수 |
| **버전** | 대부분 최신으로만 | 지금 버전 → 바꿀 버전 선택 (0.0.300 ~ 1.x, 되돌리기 포함) |

근거 데이터 · 2번

# 고칠 때 볼 근거를 미리 만들어 두었습니다

**9,155줄**

import 대응표
옛 경로 → 1.x 경로

**42개**

메서드 대응표
.run() → .invoke() 등

**295개**

공식 문서
최신 271 + 옛 가이드 24

**11,945개**

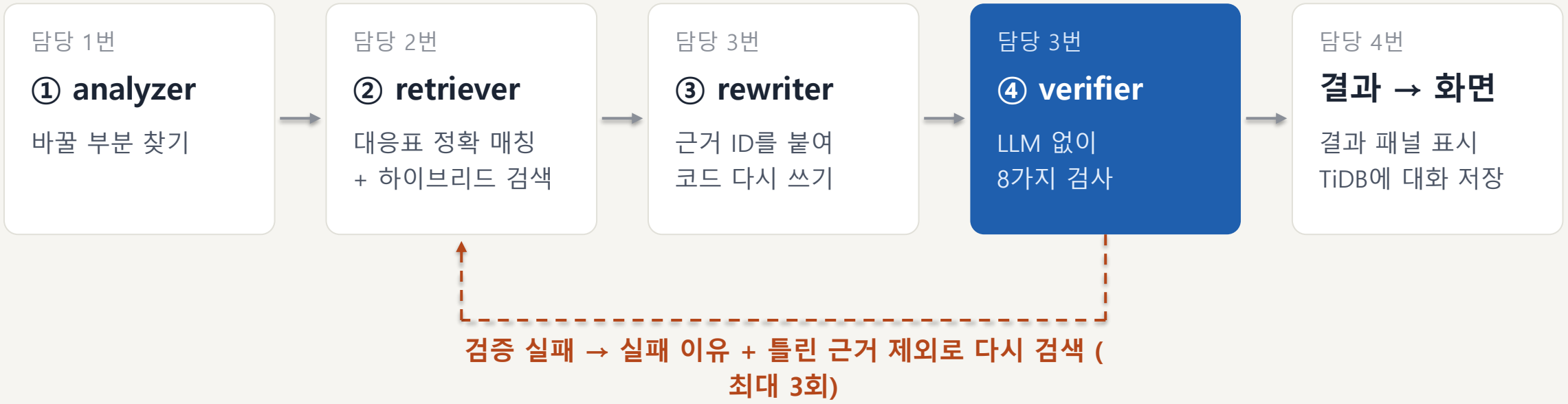
검색용 조각 (청크)
Chroma + BM25 하이브리드

## 출처 4가지

| PyPI 패키지 코드 | langchain-cli 변환 규칙 | 최신 공식 문서 | 0.3.30 옛 마이그레이션 가이드 |
|---|---|---|---|
| 버전별 도구 목록 23개 (0.0.300 ~ 1.4.4) | 공식 자동 변환 규칙 | 임베딩: text-embedding-3-small | 옛 방식 → 새 방식 설명 |

시스템 구조

# 네 단계를 거치고, 실패하면 되돌아갑니다




연결 · 재시도 판단: 1번 (LangGraph) · 변환 결과 기준: LangChain 1.4.4

검증 · 3번

# verifier는 LLM 없이 8가지를 검사합니다

**1**
### 문법
파이썬 문법 통과

**2**
### 근거 ID
이번에 보여 준 근거만 썼는지

**3**
### 변경 기록
기록과 실제 코드가 같은지

**4**
### 남은 옛 import
옛 경로가 남았는지

**5**
### import 존재
1.x에 그 경로가 진짜 있는지

**6**
### 남은 옛 메서드
.run() 같은 옛 호출 (경고)

**7**
### 안 쓰는 import
쓰지 않는 import가 생겼는지

**8**
### 반환값 보존
결과 모양이 그대로인지

오류 → 다시 시도할 이유로 돌려보냄 · 경고 → 결과에 표시만 함

화면 · 4번

# 결과와 근거를 한 화면에서 봅니다




화면 시험 캡처 · 변환은 그래프 연결 전 임시 결과, 근거 ID(imap-…)는 2번 대응표 실제 값

## 대화형 + 세션 저장

ChatGPT처럼 대화 목록, 새 대화마다 세션 하나 (TiDB)

## 버전 고르기

지금 코드 버전(자동 감지) → 바꿀 버전, 되돌리기 포함

## 결과 패널

검증 배지 · 전후 비교 · 바뀐 줄마다 근거 ID

## 예시 질문 35개

사이드바에서 골라 바로 보내기

실험 설계 · 4번

# 같은 질문 35개, 같은 조건으로 비교했습니다

### 1.x로 올리기 29개

10가지 유형: import 경로, 로더 · 분할기, 임베딩 · 벡터스토어, LLMChain → LCEL, 체인 조합, RAG, 메모리, 에이전트 · 도구, 출력 파서, 실무 상황

### 버전 오가기 6개

1.x → 0.x 되돌리기, 0.3 → 0.1 · 0.0.354, 0.0.300 → 0.2 중간 단계

### 난이도

쉬움 7 · 중간 18 · 어려움 10
35개 모두 실제로 깨지는지 미리 확인 (35/35)

| 항목 | 규칙 |
|---|---|
| **비교 대상** | 일반 LLM vs 우리 챗봇 |
| **모델** | 양쪽 같은 [모델명], temperature 0 |
| **일반 LLM 질문** | 같은 요청 문장 + 같은 코드 |
| **채점** | verifier와 같은 검사 기준 |
| **최대 시도** | 3회 (우리 챗봇) |
| **지표** | 검증 통과율 · 근거 포함률 · 재시도 효과 |

데모

# LLMChain 코드가 통과하기까지

1

## 옛 코드 입력

0.0.300 LLMChain 코드,
바꿀 버전 LangChain 1.x

2

## 1 · 2회차 검증 실패

실패 이유를 들고 근거를 다시 찾음
(틀린 근거는 제외) [실제 결과로 수정]

3

## 3회차 통과

prompt | llm | StrOutputParser()로 변환,
바뀐 줄마다 근거 ID

화면에서: 검증 배지(몇 번 만에 통과) · 전후 비교 · 변경표와 근거 ID · 대화 다시 열기

결과

# [결과 한 줄 요약: 예) 어려운 코드일수록 차이가 커졌습니다]

**우리 챗봇 · 검증 통과율**

## [__]%

**일반 LLM · 검증 통과율**

## [__]%

근거 포함률 [__]% · 질문 35개 기준

[그래프 삽입]
난이도 · 유형별 검증 통과율

검증 루프 효과

# 다시 고치는 루프가 통과율을 [+__%p] 올렸습니다

[그래프 삽입]
1회차 → 2회차 → 3회차 누적 통과율

1회차 통과율

**[__]%**

최종 통과율 (최대 3회)

**[__]%**

통과까지 평균 시도

**[__]회**

만들면서 발견

# 통과했다고 끝이 아니었습니다

## 지원 종료 중인 패키지

langchain-community는 import만 해도 '지원 종료 중' 경고를 냅니다. 지금은 통과해도 곧 깨질 수 있습니다.

langchain-community 0.4.2 확인

## classic 우회

LLMChain을 langchain_classic에서 가져오면 import는 통과하지만, 최신 방식(LCEL)으로 바꾼 것은 아닙니다.

대응표에 '옛 창고(classic)'로 따로 표시

## 옛 버전끼리는 거의 그대로

0.0.300 ~ 0.3.30 사이에는 자주 쓰는 import 경로가 거의 같습니다. 크게 바뀌는 것은 1.x와의 사이입니다.

버전별 도구 목록 비교로 확인

한계와 다음 단계

# 통과가 곧 정답은 아닙니다

| 한계 | 다음 단계 |
| --- | --- |
| import · 메서드까지 검사하지만, 실행 결과가 맞는지는 확인 못 함 | 가짜 LLM을 넣어 코드 전체를 실제로 실행 |
| 다시 쓰기(rewriter)는 1.x로 올리기 중심, 되돌리기는 화면 쪽 도구 목록 비교 | 목표 버전을 그래프 전체에 넘겨 모든 방향 지원 |
| 질문 35개, 팀이 직접 구축 | 공개 저장소의 실사용 코드로 확장 |
| LangChain만 대상 | pydantic, OpenAI SDK 등 빠르게 바뀌는 라이브러리로 확장 |

가치

# 라이브러리가 바뀔 때마다, 근거와 함께 확인된 코드

### 시간 절감

에러를 하나씩 검색하는 대신, 근거가 붙은 답을 바로 확인

### 신뢰

바뀐 줄마다 근거 ID, 몇 번 만에 검사를 통과했는지 함께 제시

### 확장

대응표 · 문서 · 검사 환경만 바꾸면 다른 라이브러리에도 적용

팀

1번 · [이름]

### 그래프 · 백엔드

LangGraph 연결 · analyzer
재시도 판단 · dev 병합

2번 · [이름]

### 데이터 · 검색

대응표 · 청킹 · Chroma
하이브리드 검색 · retriever

3번 · [이름]

### 변환 · 검증

State · rewriter
verifier 8가지 검사

4번 · [이름]

### 화면 · 평가 · 발표

Streamlit 화면 · TiDB
질문 35개 · 비교 실험

# 감사합니다

질문 받겠습니다 · github.com/SKNETWORKS-FAMILY-AICAMP/SKN35-3rd-4Team